

# TAREA — TRABAJO DE PLANEAMIENTO

## Punto de Partida de tu Proyecto

*Elegir tu proyecto y describir cómo va hasta la Sesión 9*

Desarrollo de Aplicaciones Básico

**Alumno:** [Tu nombre]

**Proyecto:** Zoapex

**Fecha:** 11 de junio de 2026

### 1. Elige tu proyecto

Mi proyecto se llama Zoapex y es una aplicación de escritorio de e-commerce para una tienda de mascotas. El prototipo se centra en mostrar el catálogo de productos y en registrar las ventas de esos productos, pero el diseño queda preparado para crecer hacia un sistema más completo. Elegí este rubro porque me permite cumplir de forma clara las tres condiciones que se aplicarán más adelante en el curso:

- **Mantenimientos CRUD:** gestiono el alta, edición y eliminación de productos y de categorías (y, en una fase posterior, de clientes).
- **Operación maestro-detalle transaccional:** el registro de un pedido, con una cabecera y varias líneas de detalle, que debe grabarse “todo o nada”. Mi pareja es Pedido → DetallePedido (en la base de datos, order → order\_detail).
- **Consultas y reportes:** el catálogo ya combina las tablas producto y categoría, y a futuro tendré un reporte de ventas que cruza pedido, detalle, producto y cliente, pensado para poder exportarse a Excel.

### 2. Describe de qué trata tu proyecto

Zoapex es un sistema para una tienda de mascotas que ordena su catálogo y registra sus ventas. Lo usan dos perfiles: el administrador, que mantiene los productos y las categorías (precios, stock, descripciones, imagen), y el usuario final o vendedor, que navega el catálogo, arma un carrito y concreta la compra. Toda la información vive en una base de datos PostgreSQL alojada en Supabase (en la nube), y la aplicación de escritorio se conecta a ella para leer y grabar datos.

La operación más importante es el registro de la venta y ocurre así, paso a paso: el usuario abre el catálogo y ve los productos traídos desde Supabase; va agregando productos al carrito indicando la cantidad de cada uno; al confirmar, el sistema valida que haya stock suficiente, calcula el subtotal, le suma el IGV del 18 % y obtiene el total; finalmente graba el pedido junto con todas sus líneas de detalle de forma transaccional y descuenta el stock de cada producto. Si cualquier paso falla, no se guarda nada, manteniendo la base de datos consistente. Por eso mi pareja maestro-detalle es Pedido → DetallePedido (order → order\_detail).

### 3. Cómo va mi proyecto hasta la Sesión 9

Apoyándome en lo visto en clase (Windows Forms y MDI, capas con CRUD en ADO.NET, reportes a Excel y la operación maestro-detalle transaccional), tengo definido el plano de mi sistema de la siguiente manera:

- **Arquitectura por capas:** el proyecto se organiza en 4 proyectos independientes. Zoapex.Entities contiene las clases POCO (es el núcleo y no referencia a nadie); Zoapex.DataAccess maneja la conexión a Supabase con el driver Npgsql y tiene una clase DAL por entidad; Zoapex.Business concentra todas las validaciones y el cálculo de totales; y Zoapex.Presentation es la interfaz en WPF. Las referencias van en una sola dirección: Presentation → Business → DataAccess → Entities, de modo que la interfaz nunca habla directo con la base de datos.
- **Tablas principales:** mis tablas maestras son category, product y customer. La pareja cabecera-detalle es order (cabecera del pedido) con order\_detail (las líneas), enlazadas para guardar la venta completa.
- **Módulos previstos:** para el administrador, el CRUD de productos y de categorías y, más adelante, el reporte de ventas exportable a Excel. Para el usuario final, el catálogo visual, el carrito de compras y el registro de la venta con descuento automático de stock.
- **Estado actual:** lo más avanzado es el diseño de la arquitectura y la base de datos: ya tengo definidas las 5 tablas, 7 funciones de PostgreSQL (equivalentes a stored procedures) y los datos de prueba. Me falta construir el código de cada capa y las ventanas en WPF, y dejo para fases siguientes el login con roles, la gestión completa de clientes y los dashboards.

*Como elementos diferenciadores, mi base de datos está en la nube (Supabase / PostgreSQL) en lugar de un SQL Server local, uso WPF en vez de Windows Forms y todo el código (clases, métodos y variables) está en inglés, dejando los comentarios en español.*

#### 4. ¿En qué tecnología hago el CRUD de administración?

Para mi proyecto elegí el esquema Form + Web. La parte administrativa (el CRUD de productos y categorías) la desarrollaré en escritorio con WPF, que en mi caso reemplaza a Windows Forms como diferenciador; y la cara pública que usará el cliente (el catálogo y la tienda) la llevaré a la web con ASP.NET Core en una fase posterior. Así, el administrador trabaja cómodo desde una app de escritorio y el cliente final puede comprar desde el navegador.

Respeto la regla de oro en todo momento: la interfaz nunca habla directo con la base de datos. Las capas Business, DataAccess y Entities se reutilizan exactamente igual en escritorio o en web, y lo único que cambia es la presentación. Por ahora esto es solo el plano: el CRUD web real lo construiré más adelante con Entity Framework y LINQ, tal como veremos en las Sesiones 12-13.